GitHub Daily Digest

2026-08-31 · Engineering report · amagarzogonzalo/llms

Executive Snapshot (from git data)

• **Repos**: amagarzogonzalo/llms

• **Commits**: 50 (refs: b147691, 892bc50, 488a587, 6e14c5a, 50481e8 (+45 more))

• **Pull requests**: 20 total, 19 merged, 1 open

• **Lines changed**: +32540 / –3928 (28612 net)

• **Files touched** (244): .env.example, .gitattributes, .github/workflows/ci.yml, .github/workflows/release.yml, .gitignore, .opencode/agents/orchestrator.md, .opencode/agents/planner.md, .opencode/agents/reviewer.md, .opencode/agents/tdd.md, .opencode/skills-managed.txt, .opencode/skills/devops-engineer/SKILL.md, .opencode/skills/devops-engineer/references/deployment-strategies.md (+232 more)

• **Active authors**: Auto CI (41), amagarzogonzalo (9)

↓ *Latest 3 merged PRs — full diff detail*

PR #28 — docs(plans): active execution plan P3→P6

Merged PR on `amagarzogonzalo/llms` - 2 file(s), +166/-1 lines.

• `docs/README.md` (modified, +3/-1)

```
@@ -27,7 +27,8 @@ docs/
■ ■■■ dev-tooling.md # Makefile philosophy, toolchain resolver, upgrade roadmap
■ ■■■ ci-cd-workflow.md # GitHub Actions: CI jobs, GPU smoke, tag-based releases
■■■ plans/
-■ ■■■ kubernetes-gpu-stacks.md # DRAFT: K8s/minikube GPU orchestration migration
+■ ■■■ kubernetes-gpu-stacks.md # DRAFT: K8s/minikube GPU orchestration migration
+■ ■■■ p3-p6-execution-plan.md # ACTIVE: phases, acceptance criteria, effort to published results
■■■ research/
■■■ README.md # research index + headline conclusions
■■■ gpu-and-model-research.md # 6GB hardware, VRAM arithmetic, model choice
@@ -53,6 +54,7 @@ docs/
| Ops | [`ops/dev-tooling.md`](ops/dev-tooling.md) | living | Makefile design, WSL↔Windows toolchain resolver, upgrade roadmap |
| Ops | [`ops/ci-cd-workflow.md`](ops/ci-cd-workflow.md) | living | CI/CD pipelines: lint/test/build, GPU smoke, tag-based releases |
| Plans | [`plans/kubernetes-gpu-stacks.md`](plans/kubernetes-gpu-stacks.md) | draft | K8s (minikube) migration for GPU stacks; scheduler-enforced engine exclusivity |
```

```
+| Plans | [`plans/p3-p6-execution-plan.md`](plans/p3-p6-execution-plan.md) | **active** |
Execution plan to published results: P3 baseline gate → P2 fine-tune → P4 sweep → P5 GGUF →
P6 second model |
```

Agent procedures (outside this tree)

- `docs/plans/p3-p6-execution-plan.md` (added, +163/-0)

```
@@ -0,0 +1,163 @@
+# Execution Plan — P3 → P6
+
+From a validated instrument (merged PRs #24–#27) to published, defensible
+results. Each phase has an objective, concrete tasks, acceptance criteria, and
+an effort estimate for the 6 GB single-GPU constraint. Sequence matters:
+**validity before training claims** (P3 gates everything).
+
+Status: `active plan` · supersedes the "next steps" checklist in
+[`training-roadmap.md`](../training-roadmap.md) §phased milestones.
+
+---
+
+## Phase 0 — Hygiene (½ day)
+
+| Task | Detail |
+|---|---|
+| Frontend async jobs | `/experiments` page: submit with `?wait=false`, poll `/jobs/{id}`
(SSE already available) |
+| Eval controls in UI | expose `subset`/`nSamples`/`seeds` fields (API supports them today) |
+| Docker default backend | flip `EVAL_EXECUTION_BACKEND=docker` after one live verification;
keep subprocess as documented dev fallback |
+| Zombie-server guard | dev script/Makefile target that kills orphaned uvicorn workers before
`make dev` |
+
+**Exit:** everything operable from the dashboard alone.
+
+---
+
+## Phase 1 — P3: Baseline sanity check (2 sessions, ~3 h GPU total)
+
+**Objective:** reproduce a published baseline with our harness before trusting
+any fine-tuned number. This is the validity gate for everything after.
+
+**Protocol (non-negotiable):**
+
+| Parameter | Value | Rationale |
+|---|---|---|
+| Model | `Qwen/Qwen2.5-Coder-1.5B-Instruct` (local) | our serving target; *note*: published
43.9% is the **base** model — record this delta explicitly |
+| Task | `humaneval`, `subset=50` first, then full `164` | runtime control |
+| n_samples | 20 | proper pass@1 sampling, not greedy |
+| Seeds | `[11, 22, 33]` | roadmap ≥3-seed protocol |
+| Temperature | 0.2 / top_p 0.95 | harness defaults |
+| Execution | Docker sandbox backend | untrusted code isolation |
+| Quantization | fp16 (serving) | held constant across all compared runs |
+
+**Tasks:**
+
+1. Pilot run: `subset=5, nSamples=2, seeds=[1]` — plumbing check (< 10 min).
+2. Main run via async API; monitor through `/jobs/{id}` SSE.
```

```
+3. Acceptance criteria:
+ - pass@1 mean within **±5 points** of the published 43.9% (Instruct-vs-base
+ delta documented if we exceed it upward);
+ - seed range ≤ 0.08 (else increase n_samples and re-run — variance means
+ the measurement, not the model, is broken);
+ - no execution-timeout storm (timeout rate < 10%).
+4. Deliverable: versioned `results/2026*_p3-baseline_humaneval.json` (auto via
+ the results ledger) + a short findings note in `docs/`.
+
+**Effort:** ~2 h GPU + ½ day analysis. **Gate:** do not start P2 before this passes.
+
+---
+
+## Phase 2 — P2: First real fine-tune (2–3 sessions)
+
+**Objective:** exercise the real 4-bit QLoRA/DoRA path on the full recipe and
+produce a servable checkpoint.
+
+| Parameter | Value |
+|---|---|
+| Base | `Qwen/Qwen2.5-Coder-1.5B-Instruct` |
+| Dataset | `datasets/recipes/pandas-codealpaca.yaml` (750 records, pinned rev) |
+| Method | `ql
```

PR #27 — docs: The Lab, Explained - DoRA, training, evaluation, statistics and the test-suite map

Merged PR on `amagarzogonzalo/llms` - 2 file(s), +363/-0 lines.

• `docs/README.md` (modified, +2/-0)

```
@@ -19,6 +19,7 @@ everything below links to the rest.

docs/
■■■ README.md ← you are here
+■■■ lab-explained.md # DoRA/QLoRA, training + evaluation, and the test suite explained
■■■ training-pipeline.md # implemented fine-tuning pipeline (stable path)
■■■ training-roadmap.md # strategy, baselines, P0–P6 milestones (stable path)
■■■ bigcode-eval-mlflow.md # BigCode harness ↔ MLflow evaluation integration
@@ -40,6 +41,7 @@ docs/

| Area | Doc | Status | Content |
|---|---|---|---|
+| Guide | [`lab-explained.md`](lab-explained.md) | living | DoRA/QLoRA, both pipelines
end-to-end, seeds/pass@k/CI statistics, the test-suite map |
| Training | [`training-pipeline.md`](training-pipeline.md) | implemented | Current
fine-tuning pipeline: QLoRA, dry-run MVP, MLflow tracking, dataset recipes |
| Training | [`training-roadmap.md`](training-roadmap.md) | active plan | Published
baselines, evaluation validity protocol, P0–P6 milestones |
| Evaluation | [`bigcode-eval-mlflow.md`](bigcode-eval-mlflow.md) | implemented |
BigCode-harness adapter, multi-seed MLflow protocol, security gate, runbooks |
```

• `docs/lab-explained.md` (added, +361/-0)

```
@@ -0,0 +1,361 @@
+# The Lab, Explained — DoRA, Training, Evaluation, and the Test Suite
+
+A plain-language but technically precise guide to everything this repository
+does: how fine-tuning works (QLoRA/DoRA), how code evaluation works (BigCode
+harness semantics, seeds, pass@k), what every test protects, and how the two
+pipelines are wired together. Written for engineers reviewing the system for
```

```
+the first time — no prior ML-ops knowledge assumed.
+
+Status: `living` · Companion docs: [`training-pipeline.md`](training-pipeline.md),
+[`bigcode-eval-mlflow.md`](bigcode-eval-mlflow.md),
[`training-roadmap.md`](training-roadmap.md).
+
+---
+
+## 1. The big picture
+
+The repository is a **hexagonal control plane**: the core (`src/core/`)
+contains only framework-free logic and *ports* (Python Protocols); everything
+that touches the real world — PyTorch, MLflow, Docker, subprocesses, HTTP —
+lives in *adapters* (`src/adapters/`) and is wired together in one place
+(`src/bootstrap/container.py`). A boundary test (`test_hexagonal_boundaries.py`)
+enforces that the core never imports adapters, so the domain logic stays
+testable without a GPU, without MLflow, and without network.
+
+```
+ HTTP API / Dashboard
+ ■
+ ■■■■■■■■■■■■■■▼■■■■■■■■■■■■■■■
+ ■ core/use_cases ■ ExperimentService, Training/Evaluation
+ ■ (pure domain logic) ■ orchestrators, multi-seed protocol
+ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
+ ■ depends only on Ports (Protocols)
+ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
+ ▼ ▼ ▼
+TrainerPort EvaluatorPort ExperimentTrackerPort / ModelRegistryPort
+ ■ ■ ■
+HuggingFace BigCode harness MLflow 3 (tracking, datasets,
+trainer adapter registry, scorers)
+```
+
+Two pipelines flow through this shape — **training** (§3) and **evaluation**
+(§4) — and one management layer orchestrates both (**ExperimentService**, §6).
+
+---
+
+## 2. DoRA and QLoRA — what the training actually does
+
+### 2.1 The problem
+
+Fine-tuning a 1.5B-parameter model the classic way means updating *every*
+weight: the weights themselves (≈3 GB in fp16), their gradients (another 3 GB)
+and optimizer state (Adam keeps two more copies ≈ 6 GB). That is ~12 GB —
+double our entire GPU. On a 6 GB RTX 4050, classic full fine-tuning is
+impossible.
+
+### 2.2 LoRA — train tiny adapters instead
+
+**LoRA** (Low-Rank Adaptation, Hu et al. 2021) freezes the base model
+completely and injects two small matrices `A` (r×in) and `B` (out×r) around
+each frozen weight matrix `W`. Only `A` and `B` are trained; the update is
+`ΔW = B·A`, scaled by `α/r`.
+
+- With rank `r = 16` and *all-linear* targeting (the 7 projection matrices of
+ each transformer block), a 1.5B model gains only **~18M trainable
+ parameters** — about 1% of the model.
- Our convention `alpha = rank` follows the Unsloth gu
```

PR #26 — feat(platform): sandboxed execution, async experiments, HTTP-first MLflow, bootstrap CIs, legacy migration

Merged PR on `amagarzogonzalo/llms` - 21 file(s), +1152/-16 lines.

• `backend/src/adapters/api/experiments.py` (modified, +80/-5)

```
@@ -2,10 +2,14 @@
training/evaluation experiments (MLflow-backed: runs, datasets, registry).
"""

-from fastapi import APIRouter, Depends, HTTPException
+import json
+
+from fastapi import APIRouter, Depends, Header, HTTPException, Query
+from fastapi.responses import JSONResponse, StreamingResponse
from starlette.concurrency import run_in_threadpool

-from src.bootstrap.container import get_experiment_service
+from src.adapters.experiment_jobs import ExperimentJobRunner
+from src.bootstrap.container import get_experiment_jobs, get_experiment_service
from src.core.entities import (
EnsureEvaluationSetsRequest,
EvaluationSetInfo,
@@ -18,7 +22,7 @@
TrainingRequest,
)
from src.core.exceptions import InvalidEvaluationError
-from src.core.use_cases.experiments import ExperimentService
+from src.core.use_cases.experiments import EvaluationEngineConfig, ExperimentService

router = APIRouter(prefix="/api/experiments", tags=["experiments"])

@@ -101,11 +105,31 @@ def _load_task_records(task: str) -> list[dict]:
return load_task_problems(task)

-@router.post("/evaluations")
+@router.post("/evaluations", response_model=None)
async def run_evaluation(
request: ExperimentEvaluationRequest,
+ wait: bool = Query(default=True),
service: ExperimentService = Depends(get_experiment_service),
+ jobs: ExperimentJobRunner = Depends(get_experiment_jobs),
) -> dict:
+ engine = EvaluationEngineConfig(
+ subset=request.subset,
+ n_samples=request.nSamples,
+ temperature=request.temperature,
+ max_new_tokens=request.maxNewTokens,
+ timeout_seconds=request.timeoutSeconds,
+ allow_code_execution=request.allowCodeExecution,
+ )
+ if not wait:
+ job = jobs.submit_evaluation(
+ model_uri=request.modelUri,
+ tasks=list(request.tasks),
+ seeds=list(request.seeds),
+ evaluator=request.evaluator,
+ run_name=request.runName,
+ engine_config=engine,
+ )
+ return JSONResponse(status_code=202, content={"jobId": job.id, "status": job.status})
```

```
try:
return await run_in_threadpool(
lambda: service.run_evaluation(
@@ -115,18 +139,31 @@ async def run_evaluation(
evaluator=request.evaluator,
run_name=request.runName,
tags=request.tags or None,
+ engine=engine,
)
)
except ValueError as exc:
raise InvalidEvaluationError(str(exc)) from exc

-@router.post("/trainings")
+@router.post("/trainings", response_model=None)
async def run_training(
request: ExperimentTrainingRequest,
+ wait: bool = Query(default=True),
service: ExperimentService = Depends(get_experiment_service),
+ jobs: ExperimentJobRunner = Depends(get_experiment_jobs),
) -> dict:
training: TrainingRequest = request.training
+ if not wait:
+ job = jobs.submit_training(
+ reque
```

- `backend/src/adapters/evaluation/execution.py` (added, +128/-0)

```
@@ -0,0 +1,128 @@
+"""Code-execution backends for grading untrusted model completions.
+
+Two implementations of the same executor contract ((program, timeout) ->
+(passed, output_tail)):
+
+- ``SubprocessCodeExecutor``: fresh host interpreter per candidate (dev
+ default; the long-standing behavior).
+- ``DockerSandboxExecutor``: each candidate runs in an ephemeral container
+ with no network, dropped capabilities, read-only rootfs, and hard
+ memory/CPU/pids limits — the professional default for broad evals.
+
+The runner callable is injectable so unit tests never touch Docker.
+"""
+
+from __future__ import annotations
+
+import subprocess # noqa: S404 - fixed argv, no shell
+import sys
+import tempfile
+from collections.abc import Callable
+from pathlib import Path
+from typing import Literal
+
+Runner = Callable[..., subprocess.CompletedProcess]
+
+DOCKER_SECURITY_FLAGS = (
+ "--rm",
+ "--network",
+ "none",
+ "--memory",
+ "512m",
+ "--cpus",
```

```
+ "1",
+ "--pids-limit",
+ "64",
+ "--cap-drop",
+ "ALL",
+ "--security-opt",
+ "no-new-privileges",
+ "--read-only",
+ "--tmpfs",
+ "/tmp:rw,size=16m",
+)
+
+
+def _write_candidate(program: str) -> Path:
+ with tempfile.NamedTemporaryFile(
+ "w", suffix=".py", encoding="utf-8", newline="\n", delete=False
+ ) as handle:
+ handle.write(program)
+ return Path(handle.name)
+
+
+class SubprocessCodeExecutor:
+ """Fresh host interpreter per candidate (dev default)."""
+
+ def __call__(self, program: str, timeout_seconds: float) -> tuple[bool, str]:
+ path = _write_candidate(program)
+ try:
+ completed = subprocess.run( # noqa: S603 - fixed argv, no shell
+ [sys.executable, str(path)],
+ capture_output=True,
+ text=True,
+ timeout=timeout_seconds,
+ check=False,
+ )
+ tail = (completed.stdout + completed.stderr)[-2000:]
+ return completed.returncode == 0, tail
+ except subprocess.TimeoutExpired:
+ return False, f"timeout after {timeout_seconds}s"
+ finally:
+ path.unlink(missing_ok=True)
+
+
+class DockerSandboxExecutor:
+ """Runs each candidate in an ephemeral locked-down container.
+
+ The candidate file is bind-mounted read-only; the container has no network,
+ no capabilities, a read-only rootfs and tight memory/CPU/pids limits.
+ """
+
+ def __init__(
+ self,
+ *,
+ image: str = "python:3.11-slim",
+ runner: Runner | None = None,
+ ) -> None:
+ self.image = image
+ self._run = runner or subprocess.run
+
+ def __call__(self, program: str, timeout_seconds: float) -> tuple[bool, str]:
+ path = _write_candidate(program)
+ container_path = "/sandbox/candidate.py"
```

```
+ argv = [
+ "docker",
+ "run",
+ *DOCKER_SECURITY_FLAGS,
+ "-v",
+
```

- `backend/src/adapters/experiment_jobs.py` (added, +123/-0)

```
@@ -0,0 +1,123 @@
+"""Async execution of experiment runs (training / evaluation) as jobs.
+
+Long fine-tunes and multi-seed evaluations must not block HTTP workers: the
+API submits a job, and the blocking experiment work runs in a worker thread
+while lifecycle events (status/result/error) stream onto the JobManager for
+polling and SSE.
+"""
+
+from __future__ import annotations
+
+import asyncio
+from typing import Any
+
+from src.core.use_cases.experiments import ExperimentService
+from src.core.use_cases.jobs import Job, JobManager
+
+
+class ExperimentJobRunner:
+ def __init__(self, *, jobs: JobManager, service: ExperimentService) -> None:
+ self.jobs = jobs
+ self.service = service
+ self._tasks: set[asyncio.Task] = set()
+
+ def submit_training(self, *, request: dict[str, Any]) -> Job:
+ job = self.jobs.create("experiment-training")
+ self._spawn(self._run_training(job.id, request))
+ return job
+
+ def submit_evaluation(
+ self,
+ *,
+ model_uri: str,
+ tasks: list[str],
+ seeds: list[int],
+ evaluator: str = "bigcode",
+ run_name: str | None = None,
+ engine_config: Any = None,
+ ) -> Job:
+ job = self.jobs.create("experiment-evaluation")
+ self._spawn(
+ self._run_evaluation(
+ job.id,
+ model_uri=model_uri,
+ tasks=tasks,
+ seeds=seeds,
+ evaluator=evaluator,
+ run_name=run_name,
+ engine_config=engine_config,
+ )
+ )
+ return job
```

```
+
+ def _spawn(self, coro) -> None:
+ task = asyncio.create_task(coro)
+ self._tasks.add(task)
+ task.add_done_callback(self._tasks.discard)
+
+ async def _run_training(self, job_id: str, request: dict[str, Any]) -> None:
+ job = self.jobs.get(job_id)
+ job.status = "running"
+ self.jobs.emit(job_id, "status", {"status": "running"})
+ try:
+ job.result = await asyncio.to_thread(self.service.run_training, **request)
+ job.status = "succeeded"
+ self.jobs.emit(job_id, "result", {"runId": _result_run_id(job.result)})
+ except Exception as exc: # noqa: BLE001 - job boundary
+ job.status = "failed"
+ self.jobs.emit(job_id, "error", {"message": str(exc)})
+ self.jobs.emit(job_id, "status", {"status": job.status})
+
+ async def _run_evaluation(
+ self,
+ job_id: str,
+ *,
+ model_uri: str,
+ tasks: list[str],
+ seeds: list[int],
+ evaluator: str,
+ run_name: str | None,
+ engine_config: Any,
+ ) -> None:
+ job = self.jobs.get(job_id)
+ job.status = "running"
+ self.jobs.emit(job_id, "status", {"status": "running"})
+ try:
+ job.result = await asyncio.to_thread(
+ lambda: self.service.run_evaluation(
+
```

- `backend/src/adapters/training/mlflow_tracker.py` (modified, +49/-3)

```
@@ -2,6 +2,7 @@
from datetime import UTC, datetime
from pathlib import Path
from tempfile import TemporaryDirectory
+from typing import Any

from src.core.entities import LoadedModelArtifact, TrainingRunRecord
from src.core.exceptions import BackendError
@@ -261,8 +262,53 @@ def _to_record(self, run) -> TrainingRunRecord:
)

-def create_experiment_tracker(tracking_uri: str, experiment_name: str) ->
ExperimentTrackerPort:
+def create_experiment_tracker(
+ tracking_uri: str,
+ experiment_name: str,
+ fallback_tracking_uri: str | None = None,
+) -> ExperimentTrackerPort:
+ """HTTP-first: prefer a reachable MLflow server (durable, UI-consistent);
+ fall back to the local SQLite store when the server is unreachable."""
try:
return MlflowExperimentTracker(tracking_uri=tracking_uri, experiment_name=experiment_name)
```

• except Exception as exc:

• return UnavailableExperimentTracker(str(exc))

```
+ except Exception:
+ if fallback_tracking_uri:
+ try:
+ return MlflowExperimentTracker(
+ tracking_uri=fallback_tracking_uri, experiment_name=experiment_name
+ )
+ except Exception as exc:
+ return UnavailableExperimentTracker(str(exc))
+ return UnavailableExperimentTracker(f"tracking server {tracking_uri} unreachable")
+
+
+def archive_legacy_experiment(
+ tracking_uri: str,
+ *,
+ source: str,
+ target: str,
+) -> dict[str, Any]:
+ """Backfills ``pipeline.run_type`` tags on every run of a legacy experiment
+ (training vs evaluation heuristics) and renames the experiment. Idempotent:
+ a missing source is a no-op."""
+ from mlflow.tracking import MlflowClient # noqa: PLC0415 - lazy, adapter-only
+
+ client = MlflowClient(tracking_uri=tracking_uri)
+ experiment = client.get_experiment_by_name(source)
+ if experiment is None:
+ return {"status": "missing", "source": source}
+
+ runs = client.search_runs([experiment.experiment_id], max_results=1000)
+ for run in runs:
+ if "train_loss" in run.data.metrics:
+ run_type = "training"
+ elif any(key.startswith(("pass@", "pass_", "tasks.")) for key in run.data.metrics) or (
+ "evaluation_tasks" in run.data.params
+ ):
+ run_type = "evaluation"
+ else:
+ run_type = "unknown"
+ client.set_tag(run.info.run_id, "pipeline.run_type", run_type)
+
+ client.rename_experiment(experiment.experiment_id, target)
+ return {"status": "archived", "source": source, "target": target, "runs_tagged": len(runs)}
```

• `backend/src/bootstrap/config.py` (modified, +9/-2)

```
@@ -1,6 +1,7 @@
import json
from functools import lru_cache
from pathlib import Path
+from typing import Literal

from pydantic_settings import BaseSettings, SettingsConfigDict

@@ -32,8 +33,10 @@ class AppConfig(BaseSettings):
model_catalog_dir: str = "data/catalog"
model_registry_file: str = "config/models.yaml"
local_models_dir: str = "~/models"
```

• # MLflow 3 requires a database backend; file store is maintenance-mode only.

• mlflow_tracking_uri: str = "sqlite:///./data/mlflow.db"

```
+ # MLflow 3: prefer the tracking server (durable, UI-consistent, artifacts
+ # served over HTTP); fall back to the local SQLite store when it is down.
+ mlflow_tracking_uri: str = "http://127.0.0.1:5000"
+ mlflow_tracking_fallback_uri: str = "sqlite:///./data/mlflow.db"
mlflow_experiment_name: str = "code-llm-lab"
```

BigCode-harness evaluation: locally staged model used for code generation.

```
bigcode_model_path: str = "data/models/Qwen2.5-Coder-1.5B-Instruct"
@@ -42,6 +45,10 @@ class AppConfig(BaseSettings):
```

Executing model-generated code is opt-in, mirroring the harness's --allow_code_execution gate.

```
bigcode_allow_code_execution: bool = False
+ # Where untrusted candidates run: host subprocess (dev) or locked-down
+ # Docker container (recommended for broad evals).
+ eval_execution_backend: Literal["subprocess", "docker"] = "subprocess"
+ eval_execution_image: str = "python:3.11-slim"
```

```
def cors_origin_list(self) -> list[str]:
raw = self.cors_origins
```

• `backend/src/bootstrap/container.py` (modified, +42/-1)

```
@@ -10,6 +10,8 @@
build_benchmark_examples,
)
from src.adapters.evaluation.bigcode import BigCodeHarnessAdapter
+from src.adapters.evaluation.execution import create_code_executor
+from src.adapters.experiment_jobs import ExperimentJobRunner
from src.adapters.providers.inference import (
HuggingFaceLocalBackend,
OllamaBackend,
@@ -27,6 +29,7 @@
from src.core.use_cases.evaluation import EvaluationOrchestrator
from src.core.use_cases.experiments import ExperimentService, MultiSeedEvaluationRunner
from src.core.use_cases.inference_router import InferenceRouter
+from src.core.use_cases.jobs import JobManager
from src.core.use_cases.multi_seed_evaluation import MultiSeedEvaluationOrchestrator
from src.core.use_cases.training.orchestrator import TrainingOrchestrator
```

```
@@ -52,10 +55,12 @@ def __init__(self, config: AppConfig, settings_service: SettingsService)
-> None
self.training_tracker = create_experiment_tracker(
tracking_uri=config.mlflow_tracking_uri,
experiment_name=f"{config.mlflow_experiment_name}-training",
+ fallback_tracking_uri=config.mlflow_tracking_fallback_uri,
)
self.evaluation_tracker = create_experiment_tracker(
tracking_uri=config.mlflow_tracking_uri,
experiment_name=f"{config.mlflow_experiment_name}-evaluation",
```

```
+ fallback_tracking_uri=config.mlflow_tracking_fallback_uri,
)
self.experiments = self.evaluation_tracker # backward-compat alias
self.model_registry = create_model_registry(config.mlflow_tracking_uri)
@@ -64,6 +69,9 @@ def __init__(self, config: AppConfig, settings_service: SettingsService) ->
None
workspace_root=Path(__file__).resolve().parents[2],
trainer=HuggingFaceTrainerAdapter(Path(__file__).resolve().parents[2]),
)
+ code_executor = create_code_executor(
+ config.eval_execution_backend, image=config.eval_execution_image
+ )
self.evaluations: dict[str, EvaluationOrchestrator] = {
"evalplus": EvaluationOrchestrator(
tracker=self.evaluation_tracker,
@@ -76,17 +84,44 @@ def __init__(self, config: AppConfig, settings_service: SettingsService)
-> None
subset=config.bigcode_subset,
n_samples=config.bigcode_n_samples,
allow_code_execution=config.bigcode_allow_code_execution,
+ executor=code_executor,
),
),
}
self.evaluation = self.evaluations["evalplus"]
+
+ def _bigcode_engine(engine_config):
+ adapter = BigCodeHarnessAdapter(
+ model_path=config.bigcode_model_path,
+ subset=engine_config.subset or config.bigcode_subset,
+ n_samples=engine_config.n_samples or config.bigcode_n_samples,
+ temperature=(
+ engine_config.temperature if engine_config.temperatu
```

- `backend/src/core/entities.py` (modified, +7/-0)

```
@@ -357,6 +357,13 @@ class ExperimentEvaluationRequest(BaseModel):
evaluator: Literal["evalplus", "bigcode"] = "bigcode"
runName: str | None = None
tags: dict[str, str] = Field(default_factory=dict)
+ # Per-request engine overrides (None -> composition-root defaults).
+ subset: int | None = Field(default=None, ge=1, le=164)
+ nSamples: int | None = Field(default=None, ge=1, le=100)
+ temperature: float | None = Field(default=None, ge=0, le=2)
+ maxNewTokens: int | None = Field(default=None, ge=16, le=4096)
+ timeoutSeconds: float | None = Field(default=None, gt=0, le=300)
+ allowCodeExecution: bool | None = None

class ExperimentTrainingRequest(BaseModel):
```

- `backend/src/core/use_cases/experiments.py` (modified, +128/-2)

```
@@ -16,11 +16,15 @@
from __future__ import annotations

import json
+import math
+import random
import subprocess # noqa: S404 - fixed argv, no shell
from collections.abc import Callable
from dataclasses import dataclass
+from datetime import UTC, datetime
```

```
from pathlib import Path
from typing import Any
+from uuid import uuid4

from src.core.entities import TrainingRequest
from src.core.ports import ExperimentTrackerPort, ModelRegistryPort
@@ -133,13 +137,52 @@ def scorer_metadata(scorers: list[ScorerInfo]) -> list[dict[str, Any]]:
BenchmarkExampleBuilder = Callable[[str, list[dict[str, Any]]], list[dict[str, Any]]]

+@dataclass(frozen=True)
+class EvaluationEngineConfig:
+ """Per-request overrides for the evaluation engine.
+
+ ``None`` fields fall back to the composition-root defaults (config). When
+ any override is set, the runner builds a dedicated engine for the request
+ instead of reusing the prebuilt one.
+ """
+
+ subset: int | None = None
+ n_samples: int | None = None
+ temperature: float | None = None
+ max_new_tokens: int | None = None
+ timeout_seconds: float | None = None
+ allow_code_execution: bool | None = None
+
+ @property
+ def has_overrides(self) -> bool:
+ return any(
+ value is not None
+ for value in (
+ self.subset,
+ self.n_samples,
+ self.temperature,
+ self.max_new_tokens,
+ self.timeout_seconds,
+ self.allow_code_execution,
+ )
+ )
+
+
class MultiSeedEvaluationRunner:
"""Selects the evaluator by name and adapts the multi-seed result into a
plain dict (including the dataset records needed for dataset registration).
+
+ ``engine_factories`` build dedicated engines for per-request overrides;
+ without overrides the prebuilt orchestrator is reused.
"""
```

- def __init__(self, runners: dict[str, Any]) -> None:

```
+ def __init__(
+ self,
+ runners: dict[str, Any],
+ engine_factories: dict[str, Callable[[EvaluationEngineConfig], Any]] | None = None,
+ ) -> None:
self.runners = runners
+ self.engine_factories = engine_factories or {}

def run(
self,
@@ -151,8 +194,15 @@ def run(
tags: dict[str, str] | None = None,
```

```
run_name: str | None = None,
evaluator: str = "bigcode",
+ engine_config: EvaluationEngineConfig | None = None,
) -> dict[str, Any]:
```

• orchestrator = self.runners.get(evaluator)

```
+ if engine_config is not None and engine_config.has_overrides:
+ factory = self.engine_factories.get(evaluator)
+ if factory is None:
+ raise ValueError(f"evaluator {evaluator!r} does not support per-request engines")
+ orchestrator = factory(engine_config)
+ else:
+ orchestrator = self.runners.get(evaluator)
if orchestrator is None:
```

• `backend/src/core/use_cases/jobs.py` (modified, +1/-0)

```
@@ -19,6 +19,7 @@ class Job:
kind: str
status: str = "queued"
events: deque[JobEvent] = field(default_factory=deque)
+ result: Any = None

class JobManager:
```

• `backend/src/core/use_cases/multi_seed_evaluation.py` (modified, +13/-1)

```
@@ -9,6 +9,7 @@

from src.core.ports import ExperimentTrackerPort
from src.core.use_cases.evaluation import EvaluationOrchestrator, extract_flat_metrics
+from src.core.use_cases.experiments import confidence_intervals

@dataclass(frozen=True)
@@ -106,13 +107,24 @@ def _aggregate(self, reports: list[dict]) -> dict[str, dict[str,
float]]:
keys = set(flat_per_seed[0])
for flat in flat_per_seed[1:]:
keys &= set(flat)
+ # Bootstrap CIs need >= 2 seeds; with 1 seed the point estimate stands alone.
+ seed_values: dict[str, list[float]] = {}
+ if len(flat_per_seed) >= 2:
+ for key in keys:
+ seed_values[key] = [flat[key] for flat in flat_per_seed]
+ intervals = confidence_intervals(seed_values)
aggregated: dict[str, dict[str, float]] = {}
for key in sorted(keys):
values = [flat[key] for flat in flat_per_seed]
```

• aggregated[key] = {

```
+ entry = {
"mean": sum(values) / len(values),
"min": min(values),
"max": max(values),
"range": max(values) - min(values),
}
+ if key in intervals:
+ low, high = intervals[key]
+ entry["ci95_low"] = low
+ entry["ci95_high"] = high
+ aggregated[key] = entry
```

```
return aggregated
```

- `backend/tests/integration/test_api_experiments.py` (modified, +28/-0)

```
@@ -178,3 +178,31 @@ async def test_reproduce_contract(fake_service):
assert ok.status_code == 200
assert ok.json()["runType"] == "training"
assert missing.status_code == 404
+
+
+async def test_async_evaluation_returns_202_and_completes(fake_service):
+ from src.adapters.experiment_jobs import ExperimentJobRunner
+ from src.bootstrap.container import get_experiment_jobs
+ from src.core.use_cases.jobs import JobManager
+
+ runner = ExperimentJobRunner(jobs=JobManager(), service=fake_service)
+ app.dependency_overrides[get_experiment_jobs] = lambda: runner
+ try:
+ async with AsyncClient(transport=ASGITransport(app=app), base_url="http://test") as client:
+ response = await client.post(
+ "/api/experiments/evaluations?wait=false",
+ json={"modelUri": "runs:/x/model", "tasks": ["humaneval"], "seeds": [1]},
+ )
+ assert response.status_code == 202
+ job_id = response.json()["jobId"]
+
+ for _ in range(200):
+ status = await client.get(f"/api/experiments/jobs/{job_id}")
+ if status.json()["status"] in {"succeeded", "failed"}:
+ break
+ body = status.json()
+ assert body["status"] == "succeeded", body["events"]
+ assert body["result"]["run_id"] == "eval-1"
+ assert fake_service.evaluations, "evaluation must have run"
+ finally:
+ app.dependency_overrides.pop(get_experiment_jobs, None)
```

- `backend/tests/unit/test_evaluation_engines.py` (added, +137/-0)

```
@@ -0,0 +1,137 @@
+"""Per-request evaluation engine controls: subset/n_samples/temperature/…
+overrides flowing from the API through the ExperimentService into per-request
+evaluator construction (create-if-needed engine factories).
+"""
+
+from typing import Any
+
+import pytest
+
+from src.core.use_cases.experiments import (
+ EvaluationEngineConfig,
+ ExperimentService,
+ MultiSeedEvaluationRunner,
+)
+
+
+class FakeTracker:
+ def start_run(self, *, run_name, params, tags=None):
+ return "run-1"
+
+ def log_metric(self, run_id, key, value, step):
```

```
+ pass
+
+ def log_json_artifact(self, run_id, artifact_path, payload):
+ self.last_artifact = (artifact_path, payload)
+
+ def log_dataset(self, run_id, **kwargs):
+ pass
+
+ def ensure_evaluation_set(self, *, name, examples, tags=None):
+ return {"name": name, "created": True, "nRecords": len(examples)}
+
+ def end_run(self, run_id, status):
+ pass
+
+ def get_run_details(self, run_id):
+ return None
+
+ def list_evaluation_sets(self):
+ return []
+
+
+class FakeOrchestrator:
+ def __init__(self, name: str):
+ self.name = name
+ self.calls: list[dict] = []
+
+ def run(self, *, model_uri, tasks, seeds, params=None, tags=None, run_name=None):
+ self.calls.append({"model_uri": model_uri, "tasks": tasks, "seeds": seeds})
+ return _result(self.name)
+
+
+def _result(name: str):
+ from src.core.use_cases.multi_seed_evaluation import MultiSeedEvaluation
+
+ return MultiSeedEvaluation(
+ run_id=f"run-{name}",
+ seeds=[1],
+ child_run_ids=[f"child-{name}"],
+ metrics={"tasks.humaneval.pass_1": {"mean": 0.5, "min": 0.5, "max": 0.5, "range": 0.0}},
+ reports=[],
+ )
+
+
+def test_runner_uses_prebuilt_orchestrator_without_engine_config():
+ prebuilt = FakeOrchestrator("prebuilt")
+ factory_calls: list[EvaluationEngineConfig] = []
+
+ def factory(config: EvaluationEngineConfig):
+ factory_calls.append(config)
+ return FakeOrchestrator("built")
+
+ runner = MultiSeedEvaluationRunner(
+ runners={"bigcode": prebuilt},
+ engine_factories={"bigcode": factory},
+ )
+ result = runner.run(model_uri="m", tasks=["humaneval"], seeds=[1])
+
+ assert factory_calls == []
+ assert result["run_id"] == "run-prebuilt"
+
+
```

```
+def test_runner_builds_engine_from_factory_when_overrides_requested():
+ prebuilt = FakeOrchestrator("prebuilt")
+ seen: list[EvaluationEngineConfig] = []
+
+ def factory(config: EvaluationEngineConfig):
+ seen.append(config)
+ return FakeOrchestrator("built")
+
+ runner = MultiSeedEvaluationRunner(
+ runners={"bigcode": prebuilt},
+ engine_factories={"bigcode": factory},
+ )
+ engine = EvaluationEngineConfig(subset=5, n_samples=10, allow_code_execution=True)
+ result = runner.run(model_uri="m", tasks=["humaneval"], seeds=[1], engine_config=engine)
```

- `backend/tests/unit/test_execution_backends.py` (added, +83/-0)

```
@@ -0,0 +1,83 @@
+"""Code-execution backend selection: host subprocess vs Docker sandbox."""
+
+import subprocess
+
+import pytest
+
+from src.adapters.evaluation.execution import (
+ DOCKER_SECURITY_FLAGS,
+ DockerSandboxExecutor,
+ SubprocessCodeExecutor,
+ create_code_executor,
+)
+
+
+def test_subprocess_executor_runs_passing_and_failing_programs():
+ execute = SubprocessCodeExecutor()
+ passed, _ = execute("print('ok')", timeout_seconds=10)
+ failed, tail = execute("import sys; sys.exit('boom')", timeout_seconds=10)
+ assert passed is True
+ assert failed is False
+ assert "boom" in tail
+
+
+def test_subprocess_executor_timeout_is_failure_not_crash():
+ execute = SubprocessCodeExecutor()
+ passed, tail = execute("import time; time.sleep(5)", timeout_seconds=1)
+ assert passed is False
+ assert "timeout" in tail
+
+
+class FakeRunner:
+ def __init__(self, returncode: int = 0):
+ self.returncode = returncode
+ self.argv: list[str] | None = None
+ self.kwargs: dict | None = None
+
+ def __call__(self, argv, **kwargs):
+ self.argv = argv
+ self.kwargs = kwargs
+ return subprocess.CompletedProcess(argv, self.returncode, stdout="", stderr="")
+
+
+def test_docker_executor_builds_locked_down_argv():
```

```
+ runner = FakeRunner()
+ execute = DockerSandboxExecutor(image="python:3.11-slim", runner=runner)
+ passed, _ = execute("print('hi')", timeout_seconds=5)
+
+ assert passed is True
+ argv = runner.argv
+ assert argv[0] == "docker" and argv[1] == "run"
+ for flag in DOCKER_SECURITY_FLAGS:
+ assert flag in argv, f"missing security flag: {flag}"
+ assert "--network none" in " ".join(argv) or (
+ argv.index("--network") + 1 < len(argv) and argv[argv.index("--network") + 1] == "none"
+ )
+ # Candidate mounted read-only into a fixed sandbox path.
+ mount = argv[argv.index("-v") + 1]
+ assert mount.endswith(":/sandbox/candidate.py:ro")
+ assert argv[-2:] == ["python", "/sandbox/candidate.py"]
+ assert runner.kwargs["timeout"] == 5
+
+
+def test_docker_executor_nonzero_exit_is_failure():
+ execute = DockerSandboxExecutor(runner=FakeRunner(returncode=1))
+ passed, _ = execute("raise SystemExit(1)", timeout_seconds=5)
+ assert passed is False
+
+
+def test_docker_executor_timeout_is_failure():
+ def hang(*args, **kwargs):
+ raise subprocess.TimeoutExpired(cmd="docker", timeout=2)
+
+ execute = DockerSandboxExecutor(runner=hang)
+ passed, tail = execute("print('x')", timeout_seconds=2)
+ assert passed is False
+ assert "timeout" in tail
+
+
+def test_create_code_executor_selects_backend():
+ assert isinstance(create_code_executor("subprocess"), SubprocessCodeExecutor)
+ assert isinstance(create_code_executor("docker"), DockerSandboxExecutor)
+ with pytest.raises(ValueError, match="Unknown execution backend"):
+ create_code_executor("firejail")
```

- `backend/tests/unit/test_experiment_jobs.py` (added, +79/-0)

```
@@ -0,0 +1,79 @@
+"""Async experiment runs: long training/evaluation jobs executed off the
+request path with lifecycle events (status/result/error) on the JobManager.
+"""
+
+import asyncio
+from typing import Any
+
+from src.adapters.experiment_jobs import ExperimentJobRunner
+from src.core.use_cases.jobs import JobManager
+
+
+class FakeService:
+ def __init__(self, *, fail: bool = False, delay: float = 0.0):
+ self.fail = fail
+ self.delay = delay
+ self.calls: list[dict[str, Any]] = []
+
```

```
+ def run_training(self, **kwargs):
+ self.calls.append(kwargs)
+ if self.delay:
+ ... # pragma: no cover
+ if self.fail:
+ raise RuntimeError("gpu exploded")
+ return {"record": {"runId": "train-1"}, "registration": {"name": "m", "version": "1"}}
+
+ def run_evaluation(self, **kwargs):
+ self.calls.append(kwargs)
+ if self.fail:
+ raise ValueError("bad request")
+ return {"run_id": "eval-1", "child_run_ids": ["eval-1"], "metrics": {}, "seeds": [1]}
+
+
+async def wait_terminal(job) -> str:
+ for _ in range(200):
+ if job.status in {"succeeded", "failed", "cancelled"}:
+ return job.status
+ await asyncio.sleep(0.01)
+ raise AssertionError("job did not reach a terminal state")
+
+
+async def test_submit_training_runs_to_completion_with_events():
+ jobs = JobManager()
+ service = FakeService()
+ runner = ExperimentJobRunner(jobs=jobs, service=service)
+
+ job = runner.submit_training(request={"training": {"runName": "r"}})
+ assert job.status in {"queued", "running"}
+
+ status = await wait_terminal(job)
+ assert status == "succeeded"
+ kinds = [event.event for event in job.events]
+ assert kinds[0] == "status"
+ assert "result" in kinds
+ assert kinds[-1] == "status"
+ assert job.result["record"]["runId"] == "train-1"
+ assert service.calls[0]["training"] == {"runName": "r"}
+
+
+async def test_submit_evaluation_reports_failure_event():
+ jobs = JobManager()
+ service = FakeService(fail=True)
+ runner = ExperimentJobRunner(jobs=jobs, service=service)
+
+ job = runner.submit_evaluation(
+ model_uri="m", tasks=["humaneval"], seeds=[1], evaluator="bigcode"
+ )
+ status = await wait_terminal(job)
+ assert status == "failed"
+ errors = [event for event in job.events if event.event == "error"]
+ assert errors and "bad request" in errors[0].data["message"]
+ assert service.calls[0]["model_uri"] == "m"
+
+
+async def test_job_result_is_attached_to_the_job_object():
+ jobs = JobManager()
+ runner = ExperimentJobRunner(jobs=jobs, service=FakeService())
+ job = runner.submit_evaluation(model_uri="m", tasks=["humaneval"], seeds=[2])
+ await wait_terminal(job)
```

```
+ assert job.result["run_id"] == "eval-1"
```

- `backend/tests/unit/test_experiments_service.py` (modified, +10/-1)

```
@@ -138,7 +138,16 @@ def __init__(self, tracker: FakeTracker):
}

def run(
```

- self, *, model_uri, tasks, seeds, params=None, tags=None, run_name=None, evaluator="bigcode"

```
+ self,
+ *,
+ model_uri,
+ tasks,
+ seeds,
+ params=None,
+ tags=None,
+ run_name=None,
+ evaluator="bigcode",
+ engine_config=None,
):
self.calls.append(
{
```

- `backend/tests/unit/test_results_ledger.py` (added, +117/-0)

```
@@ -0,0 +1,117 @@
+"""Statistical rigor + versioned results ledger (roadmap validity protocol)."""
+
+import json
+
+import pytest
+
+from src.core.use_cases.experiments import (
+ ExperimentService,
+ bootstrap_ci,
+ confidence_intervals,
+)
+from src.core.use_cases.multi_seed_evaluation import MultiSeedEvaluationOrchestrator
+
+
+class MinimalTracker:
+ def __init__(self):
+ self.runs: dict[str, dict] = {}
+ self.artifacts: list = []
+ self._n = 0
+
+ def start_run(self, *, run_name, params, tags=None):
+ self._n += 1
+ self.runs[f"r{self._n}"] = {
+ "params": params,
+ "tags": tags or {},
+ "metrics": {},
+ "artifacts": [],
+ "name": run_name,
+ }
+ return f"r{self._n}"
+
+ def log_metric(self, run_id, key, value, step):
+ self.runs[run_id]["metrics"][f"{key}.{step}"] = value
+
```

```
+ def log_json_artifact(self, run_id, artifact_path, payload):
+ self.runs[run_id]["artifacts"].append((artifact_path, payload))
+
+ def end_run(self, run_id, status):
+ pass
+
+ def log_dataset(self, run_id, **kwargs):
+ pass
+
+
+class FixedEvaluator:
+ def __init__(self, passes_by_seed):
+ self.passes_by_seed = passes_by_seed
+
+ def evaluate(self, model_uri, tasks, seeds):
+ (seed,) = seeds
+ return {
+ "tasks": {"humaneval": {"pass@1": self.passes_by_seed[seed]}},
+ "dataset": {},
+ }
+
+
+def test_bootstrap_ci_is_centered_and_bounded():
+ values = [0.4, 0.5, 0.6, 0.45, 0.55]
+ low, high = bootstrap_ci(values, iterations=500, seed=7)
+ assert low <= min(values) <= high or low <= 0.5 <= high
+ assert low <= high
+ assert 0.0 <= low and high <= 1.0
+
+
+def test_bootstrap_ci_is_deterministic_for_a_seed():
+ values = [0.3, 0.5, 0.7]
+ assert bootstrap_ci(values, seed=3) == bootstrap_ci(values, seed=3)
+
+
+def test_confidence_intervals_format_keys():
+ flat = {"tasks.humaneval.pass_1": [0.5, 0.6]}
+ cis = confidence_intervals(flat)
+ assert cis["tasks.humaneval.pass_1"] == pytest.approx((0.5, 0.6), abs=0.2)
+
+
+def test_multi_seed_aggregate_includes_ci95():
+ tracker = MinimalTracker()
+ evaluator = FixedEvaluator({1: 0.4, 2: 0.6})
+ from src.core.use_cases.evaluation import EvaluationOrchestrator
+
+ multi = MultiSeedEvaluationOrchestrator(
+ tracker=tracker,
+ orchestrator=EvaluationOrchestrator(tracker=tracker, evaluator=evaluator),
+ )
+ result = multi.run(model_uri="m", tasks=["humaneval"], seeds=[1, 2])
+ aggregate = result.metrics["tasks.humaneval.pass_1"]
+ assert aggregate["ci95_low"] <= 0.5 <= aggregate["ci95_high"]
+
+
+def test_record_result_writes_versioned_json(tmp_path):
+ class NullRegistry:
+ pass
+
+ service = ExperimentService(
+ training_tracker=MinimalTracker(),
```

```
+ evaluation_tracker=MinimalTracker(),
+ training_orchestrator=None,
+
```

- `backend/tests/unit/test_training_tracker.py` (modified, +72/-0)

```
@@ -23,6 +23,78 @@ def test_unavailable_tracker_raises_backend_error_for_all_operations():
tracker.get_run_details("r")

+def test_factory_falls_back_to_sqlite_when_http_unavailable(monkeypatch, tmp_path):
+ from src.adapters.training import mlflow_tracker
+
+ calls: list[str] = []
+
+ class FakeTracker:
+ def __init__(self, *, tracking_uri, experiment_name):
+ calls.append(tracking_uri)
+ if tracking_uri.startswith("http"):
+ raise ConnectionError("mlflow server down")
+ self.tracking_uri = tracking_uri
+
+ monkeypatch.setattr(mlflow_tracker, "MlflowExperimentTracker", FakeTracker)
+ tracker = mlflow_tracker.create_experiment_tracker(
+ "http://127.0.0.1:5000",
+ "exp",
+ fallback_tracking_uri=f"sqlite:///{tmp_path}/mlflow.db",
+ )
+ assert isinstance(tracker, FakeTracker)
+ assert calls == ["http://127.0.0.1:5000", f"sqlite:///{tmp_path}/mlflow.db"]
+
+
+def test_factory_returns_unavailable_when_every_uri_fails(monkeypatch):
+ from src.adapters.training import mlflow_tracker
+
+ class FakeTracker:
+ def __init__(self, *, tracking_uri, experiment_name):
+ raise ConnectionError("down")
+
+ monkeypatch.setattr(mlflow_tracker, "MlflowExperimentTracker", FakeTracker)
+ tracker = mlflow_tracker.create_experiment_tracker(
+ "http://127.0.0.1:5000", "exp", fallback_tracking_uri="sqlite:///./data/mlflow.db"
+ )
+ assert isinstance(tracker, mlflow_tracker.UnavailableExperimentTracker)
+
+
+def test_archive_legacy_experiment_backfills_run_types_and_renames(tmp_path):
+ from src.adapters.training.mlflow_tracker import (
+ MlflowExperimentTracker,
+ archive_legacy_experiment,
+ )
+
+ uri = f"sqlite:///{tmp_path}/mlflow.db"
+ tracker = MlflowExperimentTracker(tracking_uri=uri, experiment_name="legacy")
+ tracker.log_metric(tracker.start_run(run_name="t", params={}), "train_loss", 1.0, step=0)
+ eval_run = tracker.start_run(run_name="e", params={"evaluation_tasks": "humaneval"})
+ tracker.log_metric(eval_run, "tasks.humaneval.pass_1", 0.5, step=0)
+ tracker.start_run(run_name="u", params={})
+
+ summary = archive_legacy_experiment(uri, source="legacy", target="legacy-archived")
+ assert summary["status"] == "archived"
+ assert summary["runs_tagged"] == 3
```

```
+
+ client = tracker.client
+ names = {e.name: e.experiment_id for e in client.search_experiments()}
+ assert "legacy" not in names
+ assert "legacy-archived" in names
+
+ runs = client.search_runs([names["legacy-archived"]], max_results=10)
+ tagged = {r.data.tags.get("pipeline.run_type") for r in runs}
+ assert tagged == {"training", "evaluation", "unknown"}
+
+
+def test_archive_legacy_experiment_is_noop_for_missing_source(tmp_path):
+ from src.adapters.training.mlflow_tracker import archive_legacy_experiment
+
+ summary = archive_legacy_experiment(
+ f"sqlite:///{tmp_path}/
```

- `docs/bigcode-eval-mlflow.md` (modified, +33/-0)

```
@@ -135,6 +135,39 @@ Two scorers are implemented and registered (`GET
/api/experiments/scorers`):
Every evaluation run stores the resolved scorer metadata as
`evaluation/scorers.json`, making scoring auditable per run.
```

```
+## Platform hardening (this branch)
+
+- **HTTP-first MLflow**: the backend logs to `http://127.0.0.1:5000` (the UI
+ server, which now `--serve-artifacts`) and falls back to the local SQLite
+ store when the server is down. The server owns the DB file, so the
+ Docker-Desktop bind-mount staleness is gone: new runs appear in the UI
+ immediately, no container recreation needed.
+- **Docker-sandboxed execution**: `EVAL_EXECUTION_BACKEND=docker` runs every
+ untrusted candidate in an ephemeral `python:3.11-slim` container with no
+ network, read-only rootfs, dropped capabilities and memory/CPU/pids caps
+ (`adapters/evaluation/execution.py`). Default remains host-subprocess for
+ dev; set docker for broad evals.
+- **Per-request engine overrides**: `POST /api/experiments/evaluations`
+ accepts `subset`, `nSamples`, `temperature`, `maxNewTokens`,
+ `timeoutSeconds`, `allowCodeExecution` — a dedicated engine is built per
+ request; without overrides the prebuilt one is reused.
+- **Async runs**: `POST .../trainings|evaluations?wait=false` returns
+ `202 {jobId}`; poll `GET /api/experiments/jobs/{id}` or stream
+ `GET /api/experiments/jobs/{id}/events` (SSE, Last-Event-ID resumable).
+- **Bootstrap CIs**: multi-seed aggregates report `ci95_low`/`ci95_high`
+ (percentile bootstrap over per-seed values, deterministic seed).
+- **Results ledger**: evaluation runs append versioned JSON records
+ (`created_at`, `vcs_commit`, metrics) to `backend/data/results/` — records
+ are never overwritten.
+- **Legacy migration**: `archive_legacy_experiment()` backfills
+ `pipeline.run_type` tags (train_loss → training, pass@/tasks.* →
+ evaluation) and renames the pre-split experiment to
+ `code-llm-lab-legacy`.
+- **Toolchain guard**: `scripts/dev/tool.sh` now prefers the Windows-side
+ `.venv` tools through WSL interop when the checkout is a `/mnt` mount and a
+ Windows venv exists — `make verify` can no longer rebuild the venv for the
+ wrong platform.
+
```

Environment notes (GPU box, Windows host)

• `uv sync --extra dev --extra evaluation --extra training` installs the

• `scripts/dev/lib.sh` (modified, +10/-0)

```
@@ -88,6 +88,16 @@ tail_log() {
```

otherwise the Windows .exe through WSL interop.

```
tool_cmd() {
local name="$1"
+ local dir="${2:-}"
+ # A Windows-side .venv (Scripts/python.exe) must never be touched by native
+ # WSL uv/npm: prefer the .exe through interop when one already exists.
+ if [[ -n "$dir" ]] && is_windows_mount && [[ -f "$dir/.venv/Scripts/python.exe" ]]; then
+ require_command cmd.exe || return 1
+ if command -v "${name}.exe" >/dev/null 2>&1; then
+ printf 'cmd.exe /d /s /c %s.exe' "$name"
+ return 0
+ fi
+ fi
if command -v "$name" >/dev/null 2>&1; then
printf '%s' "$name"
return 0
```

• `scripts/dev/tool.sh` (modified, +1/-1)

```
@@ -32,7 +32,7 @@ case "$app" in
*) fail "Unknown app '$app' (expected: backend or frontend)."; usage 1 ;;
esac

-resolved="$(tool_cmd "$tool")"
+resolved="$(tool_cmd "$tool" "$dir")"
cd "$dir"
```

shellcheck disable=SC2086 # resolved is an intentional word-split command

```
exec $resolved "$@"
```

• `stacks/mlflow/docker-compose.yml` (modified, +2/-0)

```
@@ -21,6 +21,8 @@ services:
--port 5000
--backend-store-uri sqlite:////mlflow/data/mlflow.db
--default-artifact-root file:///mlflow/runs
+ --serve-artifacts
+ --artifacts-destination /mlflow/runs
healthcheck:
test: ["CMD", "python3", "-c", "import urllib.request;
urllib.request.urlopen('http://localhost:5000/health')"]
interval: 10s
```

Other merged PRs

• PR #25: feat(evaluation): BigCode harness adapter + multi-seed MLflow evaluation protocol [44 file(s), +4638/-95]

• PR #24: fix(runtime): Proactor subprocess fallback + Qwen2.5-Coder local profile [41 file(s), +3337/-126]

• PR #22: feat(evaluation): MLflow-tracked local HumanEval baseline (P1 sanity-check harness) [37 file(s), +3239/-125]

• PR #21: perf(backend): keep the event loop responsive under training, eval and telemetry load [13 file(s), +332/-41]

• PR #20: fix(frontend): deploy adapter formatting + telemetry cadence tuning [6 file(s), +20/-12]

• PR #19: docs(research): 6GB VRAM GPU & model research + literature database [10 file(s), +1458/-12]

• PR #18: feat(make): ollama-pull — populate the provider model volume [5 file(s), +33/-4]

• PR #17: docs(agents): mandatory PR-only workflow for main [1 file(s), +11/-0]

• PR #15: fix(frontend): same-origin /api proxy for production dashboard

• PR #14: feat(dev): production dashboard image, env-driven compose, parallel-worktree support [11 file(s), +588/-64]

• PR #13: Feature/ci optimizations [2 file(s), +12/-5]

• PR #12: feat: add CI/CD enhancements - caching, release workflow, deployment [4 file(s), +113/-7]

• PR #11: docs(frontend): agent contract for opencode, Lovable demoted [1 file(s), +27/-12]

• PR #10: chore(make): complete make workflow for backend, frontend, and CI parity [87 file(s), +12044/-3099]

• PR #9: fix(backend): speed up provider runtime probes and GPU telemetry [100 file(s), +4887/-275]

• PR #8: fix(backend): cache runtime snapshots and use IPv4 loopback for provider probes [4 file(s), +32/-7]

Produced from git data; no AI involved. Diff hunks are verbatim from the repository.

Daily Digest (AI summary)

Detailed PR Analysis

PR #28 — docs(plans): active execution plan P3→P6

Overview

This pull request introduces a formal, active execution plan (`docs/plans/p3-p6-execution-plan.md`) governing the transition from a validated instrumentation layer (PRs #24–#27) to published, defensible empirical results under strict hardware constraints. Simultaneously, it updates the repository's central documentation index (`docs/README.md`) to surface this plan as the primary operational roadmap for phases P3 through P6, superseding previous ad-hoc milestones.

```
---
```

Significant File Analysis

1. `docs/README.md`

* **What changed:** The documentation index was updated in two places: the directory tree structure overview and the primary living documents markdown table.

* **Key identifiers added:** A new file reference `docs/plans/p3-p6-execution-plan.md` mapped with the active description status.

* **Architectural impact:** Establishes clear discoverability for the new execution roadmap, explicitly linking the project's plan index to the P3 baseline gate through P6 second-model evaluation phases.

```
-■ ■■■ kubernetes-gpu-stacks.md # DRAFT: K8s/minikube GPU orchestration migration
+■ ■■■ kubernetes-gpu-stacks.md # DRAFT: K8s/minikube GPU orchestration migration
+■ ■■■ p3-p6-execution-plan.md # ACTIVE: phases, acceptance criteria, effort to published
results
```

```
+| Plans | [`plans/p3-p6-execution-plan.md`](plans/p3-p6-execution-plan.md) | **active** |
Execution plan to published results: P3 baseline gate → P2 fine-tune → P4 sweep → P5 GGUF →
P6 second model |
```

2. `docs/plans/p3-p6-execution-plan.md`

* **What changed:** Added a comprehensive, 163-line execution plan detailing rigorous validation gates, operational hygiene tasks, and structured protocols for single-GPU (6GB VRAM) execution.

* **Key identifiers & Protocols:**

```
* Targets the **6 GB single-GPU constraint** across sequential phases.
* Explicitly mandates validity before training claims, designating **Phase 1 (P3: Baseline
sanity check)** as a non-negotiable blocking gate.
* Formulates precise protocols for model parameters, runtime task arguments, and validation
tolerances:
```

```
+# Execution Plan — P3 → P6
+
+From a validated instrument (merged PRs #24–#27) to published, defensible
+results. Each phase has an objective, concrete tasks, acceptance criteria, and
+an effort estimate for the 6 GB single-GPU constraint. Sequence matters:
+**validity before training claims** (P3 gates everything).
```

* **Operational Hygiene (Phase 0):** Outlines concrete UI and backend task structures to bridge API capabilities with the dashboard interface before running benchmarks:

```
+## Phase 0 — Hygiene (½ day)
+
+| Task | Detail |
```

|---|---|

```
+| Frontend async jobs | `/experiments` page: submit with `?wait=false`, poll `/jobs/{id}`
(SSE already available) |
+| Eval controls in UI | expose `subset`/`nSamples`/`seeds` fields (API supports them today) |
+| Docker default backend | flip `EVAL_EXECUTION_BACKEND=docker` after one live verification;
keep subprocess as documented dev fallback |
+| Zombie-server guard | dev script/Makefile target that kills orphaned uvicorn workers before
`make dev` |
```

* **Rigorous Baseline Protocol (Phase 1/P3):** Defines a strict testing harness configuration to reproduce published figures and bound experimental variance:

```
+| Parameter | Value | Rationale |
+|---|---|---|
+| Model | `Qwen/Qwen2.5-Coder-1.5B-Instruct` (local) | our serving target; *note*: published
43.9% is the **base** model — record this delta explicitly |
+| Task | `humaneval`, `subset=50` first, then full `164` | runtime control |
```

```
+| n_samples | 20 | proper pass@1 sampling, not greedy |
+| Seeds | `[11, 22, 33]` | roadmap ≥3-seed protocol |
+| Temperature | 0.2 / top_p 0.95 | harness defaults |
+| Execution | Docker sandbox backend | untrusted code isolation |
+| Quantization | fp16 (serving) | held constant across all compared runs |
```

* **Acceptance Criteria & Gates:** Enforces strict statistical bounds and error control parameters before downstream phases (such as fine-tuning) can commence:

```
+3. Acceptance criteria:
+ - pass@1 mean within **±5 points** of the published 43.9% (Instruct-vs-base
+ delta documented if we exceed it upward);
+ - seed range ≤ 0.08 (else increase n_samples and re-run — variance means
+ the measurement, not the model, is broken);
+ - no execution-timeout storm (timeout rate < 10%).
```

```
---
```

Risks, Tradeoffs, and Integration Touch-Points

* **Blocking Dependency (P3 Gate):** The plan intentionally establishes Phase 1 (P3) as an absolute blocker (`Gate: do not start P2 before this passes`). This mitigates the risk of optimizing or fine-tuning models atop a flawed or uncalibrated evaluation harness.

* **Hardware Constraint Management:** By explicitly bounding tasks to the 6GB single-GPU target, the plan forces practical runtime choices (e.g., subset sizes, `n_samples=20`, FP16 serving constraints, Docker isolation) that fit local memory limits without sacrificing statistical significance.

* **UI-API Alignment:** Phase 0 highlights an existing drift where the backend API supports parameters (`subset`, `nSamples`, `seeds`) and asynchronous polling (`/jobs/{id}` via SSE) that are not yet fully surfaced or exposed in the `/experiments` frontend. Closing this gap is treated as a prerequisite for clean execution tracking.

PR #27 — docs: The Lab, Explained - DoRA, training, evaluation, statistics and the test-suite map

Overview

PR #27 introduces a comprehensive, foundational documentation artifact (`docs/lab-explained.md`) and updates the documentation index (`docs/README.md`) to map it. The intent is to provide engineers reviewing the system for the first time with a plain-language yet technically rigorous walkthrough of the repository's hexagonal architecture, QLoRA/DoRA fine-tuning mechanics, evaluation pipelines, statistical validation (seeds/`pass@k`), and the complete test-suite layout without requiring prior MLOps domain knowledge.

```
---
```

Detailed File Analysis

1. `docs/README.md`

* **What Changed:** Added a file reference for the new guide under the directory layout ASCII tree and integrated a row entry for it in the central documentation catalog table.

* **Key Identifiers & Content Integration:**

```
* Tree path: `docs/lab-explained.md`
* Table metadata: Guide category, pointing to `lab-explained.md`, marked as `living` status.
```

* **Architectural Impact:** Exposes the newly synthesized architecture and training/evaluation blueprint directly from the repository's root entrypoint documentation, ensuring engineers discover the cross-cutting reference immediately.

* **Verbatim Patch Snippets:**

```
docs/
■■■ README.md ← you are here
+■■■ lab-explained.md # DoRA/QLoRA, training + evaluation, and the test suite explained
■■■ training-pipeline.md # implemented fine-tuning pipeline (stable path)
```

```
| Area | Doc | Status | Content |
|---|---|---|---|
+| Guide | [`lab-explained.md`](lab-explained.md) | living | DoRA/QLoRA, both pipelines
end-to-end, seeds/pass@k/CI statistics, the test-suite map |
| Training | [`training-pipeline.md`](training-pipeline.md) | implemented | Current
fine-tuning pipeline: QLoRA, dry-run MVP, MLflow tracking, dataset recipes |
```

2. `docs/lab-explained.md` (New File, +361 lines)

* **What Changed:** Added a 361-line architectural, algorithmic, and operational master document explaining the system design end-to-end.

* **Key Sections & Architectural Impact:**

```
* **Hexagonal Control Plane (§1):** Documents the separation of core business logic from outer
infrastructure via ports (Python Protocols) and adapters. Explicitly defines the dependency
inversion rule enforced by architectural boundary tests:
```

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ core/use_cases ■ ExperimentService, Training/Evaluation
■ (pure domain logic) ■ orchestrators, multi-seed protocol
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ depends only on Ports (Protocols)
■■■■■■■■■■■■■■■■■■■■■■■■
... ...
```

```
* **DoRA & QLoRA Mechanics (§2):** Breaks down the memory constraints of 1.5B-parameter models
on constrained hardware (e.g., 6 GB RTX 4050), illustrating how LoRA freezes base weights
(`W`) and trains low-rank adapter matrices (`A` and `B`), while DoRA (Weight-Decomposed
Low-Rank Adaptation) decouples magnitude scaling from directional updates.
* **Training & Evaluation Pipelines (§3 & §4):** Documents the orchestration flow between
`TrainerPort`, `EvaluatorPort`, and `ExperimentTrackerPort` interacting with Hugging Face,
the BigCode evaluation harness, and MLflow 3.
* **Test-Suite Map:** Establishes the mapping of tests that protect hexagonal boundaries,
training runs, and statistical validity.
```

* **Verbatim Patch Snippets:**

```
The repository is a **hexagonal control plane**: the core (`src/core/`)
contains only framework-free logic and *ports* (Python Protocols); everything
that touches the real world — PyTorch, MLflow, Docker, subprocesses, HTTP —
lives in *adapters* (`src/adapters/`) and is wired together in one place
(`src/bootstrap/container.py`). A boundary test (`test_hexagonal_boundaries.py`)
enforces that the core never imports adapters, so the domain logic stays
testable without a GPU, without MLflow, and without network.
```

```
---
```

Risks, Tradeoffs, and Integration Touch-Points

* **Documentation Drift:** As a `living` document mapping core domain layers, hexagonal ports, and training parameters, `lab-explained.md` risks diverging from implementation details if changes are made to `src/core/` or `src/bootstrap/container.py` without synchronized doc updates.

* **Integration Touch-Points:** The document tightly couples conceptual explanations with the actual codebase layout—specifically referencing `src/core/use_cases`, `src/adapters/`, `src/bootstrap/container.py`, and `test_hexagonal_boundaries.py`. Any refactoring of these paths will require updates to this guide.

* **Tradeoff:** Adding a large narrative markdown file increases maintenance overhead for future PRs touching architectural boundaries, but significantly lowers the onboarding barrier for engineers reviewing the repository's domain logic and hardware optimization choices (QLoRA/DoRA on constrained GPUs).

PR #26 — feat(platform): sandboxed execution, async experiments, HTTP-first MLflow, bootstrap CIs, legacy migration

This pull request introduces critical platform hardening for the MLflow-backed code-LLM experimentation engine, adding robust sandboxed code execution via Docker, asynchronous non-blocking job submissions, HTTP-first tracking fallbacks, statistical rigor through bootstrap confidence intervals, and legacy experiment migration. These additions decouple heavy training and multi-seed evaluation pipelines from blocking HTTP request lifecycles while hardening the evaluation grading pipeline against untrusted model completions.

```
---
```

File-by-File Architectural Analysis

1. `backend/src/adapters/evaluation/execution.py` (Added)

* **What changed:** Implements code-execution backends to grade untrusted model completions safely, providing both a host-level subprocess execution path and a hardened ephemeral Docker container sandbox.

* **Key identifiers & functions:**

```
* `DOCKER_SECURITY_FLAGS`: A tuple of strict container confinement settings.
* `_write_candidate(program: str) -> Path`: Writes model-generated Python code into a
temporary file on the host (`suffix=".py"`, `newline="\n"`, `delete=False`).
* `SubprocessCodeExecutor`: A development default running a fresh host interpreter per
candidate.
* `DockerSandboxExecutor`: A production-grade sandbox enforcing resource caps, dropping all
capabilities, and mounting code read-only.
```

* **Architectural Impact:** Allows platform administrators to isolate execution of arbitrary model-generated Python code, completely preventing host filesystem or network compromise during benchmark grading.

* **Key Code Snippet:**

```
DOCKER_SECURITY_FLAGS = (
"--rm",
"--network",
"none",
"--memory",
"512m",
"--cpus",
"1",
```

```
"--pids-limit",
"64",
"--cap-drop",
"ALL",
"--security-opt",
"no-new-privileges",
"--read-only",
"--tmpfs",
"/tmp:rw,size=16m",
)
```

2. `backend/src/adapters/experiment_jobs.py` (Added)

* **What changed:** Introduces an asynchronous job runner pattern to offload long-running experiment workflows (training and evaluation) away from synchronous API worker threads.

* **Key identifiers & classes:**

```
* `ExperimentJobRunner`: Orchestrates training and evaluation jobs using background
`asyncio.Task` creation.
* `submit_training` / `submit_evaluation`: Instantiates tracked job entries via `JobManager`
and dispatches coroutines asynchronously via `_spawn()`.
```

* **Architectural Impact:** Eliminates HTTP gateway timeouts on multi-seed evaluations and extensive fine-tuning runs. Lifecycle events can be polled or streamed via Server-Sent Events (SSE).

* **Key Code Snippet:**

```
class ExperimentJobRunner:
def __init__(self, *, jobs: JobManager, service: ExperimentService) -> None:
self.jobs = jobs
self.service = service
self._tasks: set[asyncio.Task] = set()

def submit_training(self, *, request: dict[str, Any]) -> Job:
job = self.jobs.create("experiment-training")
self._spawn(self._run_training(job.id, request))
return job
```

3. `backend/src/adapters/api/experiments.py` (Modified)

* **What changed:** Upgraded the evaluation execution endpoint to support conditional asynchronous execution (`wait=false`) and per-request configuration overrides.

* **Key identifiers & endpoints:**

```
* `@router.post("/evaluations", response_model=None)`: Accepts query parameter `wait: bool`
and maps request parameters to a fresh `EvaluationEngineConfig`.
* Returns an HTTP `202 Accepted` response with a `jobId` and `status` when `wait=false`.
```

* **Architectural Impact:** Direct translation of client requests into background workers, aligning the FastAPI layer with asynchronous job semantics.

* **Key Code Snippet:**

```
@router.post("/evaluations", response_model=None)
async def run_evaluation(
request: ExperimentEvaluationRequest,
wait: bool = Query(default=True),
service: ExperimentService = Depends(get_experiment_service),
```

```
jobs: ExperimentJobRunner = Depends(get_experiment_jobs),
) -> dict:
engine = EvaluationEngineConfig(
subset=request.subset,
n_samples=request.nSamples,
temperature=request.temperature,
max_new_tokens=request.maxNewTokens,
timeout_seconds=request.timeoutSeconds,
allow_code_execution=request.allowCodeExecution,
)
if not wait:
job = jobs.submit_evaluation(...)
return JSONResponse(status_code=202, content={"jobId": job.id, "status": job.status})
```

4. `backend/src/adapters/training/mlflow_tracker.py` (Modified)

* **What changed:** Added HTTP-first MLflow tracker initialization with local SQLite failover, along with utility logic to migrate legacy experiments.

* **Key functions:**

```
* `create_experiment_tracker(tracking_uri, experiment_name, fallback_tracking_uri=None)`:
Attempts connection to the primary HTTP tracking server, falling back to a local database upon
connection failure.
* `archive_legacy_experiment(tracking_uri, *, source, target)`: Backfills pipeline run-type
tags (`training` vs `evaluation`) and handles experiment renaming idempotently.
```

* **Architectural Impact:** Resiliency improvements guarantee that temporary downtime of the centralized MLflow server does not crash training/evaluation tracking nodes.

* **Key Code Snippet:**

```
def create_experiment_tracker(
tracking_uri: str,
experiment_name: str,
fallback_tracking_uri: str | None = None,
) -> ExperimentTrackerPort:
"""HTTP-first: prefer a reachable MLflow server (durable, UI-consistent);
fall back to the local SQLite store when the server is unreachable."""
try:
return MlflowExperimentTracker(tracking_uri=tracking_uri, experiment_name=experiment_name)
except Exception:
if fallback_tracking_uri:
try:
return MlflowExperimentTracker(
tracking_uri=fallback_tracking_uri, experiment_name=experiment_name
)
except Exception as exc:
return UnavailableExperimentTracker(str(exc))
return UnavailableExperimentTracker(f"tracking server {tracking_uri} unreachable")
```

5. `backend/src/bootstrap/config.py` & `container.py` (Modified)

* **What changed:** Updated application configuration settings to establish HTTP-first MLflow URIs (`http://127.0.0.1:5000`), SQLite fallback URIs, and code execution parameters (`eval_execution_backend`, `eval_execution_image`). The DI container was adapted to bind code executors and experiment job runners.

* **Architectural Impact:** Centralizes environment configuration properties and provides modular dependency injection for execution adapters.

* **Key Code Snippet:**

```
mlflow_tracking_uri: str = "http://127.0.0.1:5000"
mlflow_tracking_fallback_uri: str = "sqlite:///./data/mlflow.db"
eval_execution_backend: Literal["subprocess", "docker"] = "subprocess"
eval_execution_image: str = "python:3.11-slim"
```

6. `backend/src/core/use_cases/multi_seed_evaluation.py` & `experiments.py` (Modified)

* **What changed:** Implemented `EvaluationEngineConfig` for dynamic per-request engine overrides and integrated percentile bootstrap confidence intervals (`ci95_low`, `ci95_high`) for multi-seed evaluations.

* **Architectural Impact:** Empowers users to tune evaluation run properties (e.g., subset, temperature, sample counts) per request while boosting statistical validity across seeds.

* **Key Code Snippet:**

```
seed_values: dict[str, list[float]] = {}
if len(flat_per_seed) >= 2:
for key in keys:
seed_values[key] = [flat[key] for flat in flat_per_seed]
intervals = confidence_intervals(seed_values)
```

7. Build/Dev Tooling (`scripts/dev/lib.sh` & `scripts/dev/tool.sh`)

* **What changed:** Enhanced `tool_cmd` to detect Windows-side virtual environments (`.venv/Scripts/python.exe`) via WSL interop when running inside a Windows mount directory (`/mnt`).

* **Architectural Impact:** Prevents accidental recreation of virtual environments targeting mismatched host platforms during local `make verify` cycles.

```
---
```

Risks, Tradeoffs, and Integration Touch-Points

1. **Docker Daemon Dependency:** Utilizing `eval_execution_backend = "docker"` requires a running Docker daemon accessible by the application runtime. Environments lacking Docker must rely on the default host subprocess sandbox.

2. **Background Task Tracking:** The `ExperimentJobRunner` retains active tasks in a local `set[asyncio.Task]`. In long-running multi-process configurations, job state storage relies entirely on in-memory `JobManager` structures unless backed by a persistent data store.

3. **Network Isolation Tradeoff:** `DockerSandboxExecutor` sets `--network none`, ensuring complete isolation for untrusted code execution. Benchmarks requiring package installation or network fetching during candidate execution will fail unless mocked or pre-baked into `eval_execution_image`.

Overall

Overall, recent repository activity has advanced the llms project into a hardened, production-grade control plane for fine-tuning and evaluating small code models like Qwen2.5-Coder-1.5B under a strict 6GB VRAM constraint. Major capabilities added include an integrated BigCode evaluation harness with multi-seed bootstrapping and MLflow tracking, Docker-sandboxed code execution backends, and asynchronous experiment job runners with SSE event streaming. Additionally, the platform now features a unified

experiment management UI, comprehensive hardware/methodology research documentation, and a structured execution plan for phases P3 through P6. Primary watch-outs for subsequent development involve managing resource contention and process isolation during parallel sandbox executions, ensuring strict adherence to hexagonal boundaries as new adapter layers expand, and maintaining synchronization between local disk models and the MLflow artifact store.

Executive Overview

This digest documents the production hardening of the local LLM experimentation and evaluation engine, optimized for small code models (specifically targeting **Qwen2.5-Coder-1.5B**) under a strict **6GB VRAM** single-GPU constraint (RTX 4050, WDDM/WSL2). Through PRs `#26`, `#27`, and `#28`, the architecture enforces strict hexagonal boundaries, isolated container execution for grading, asynchronous experiment orchestration, and statistical validation protocols (percentile bootstrap confidence intervals).

Under the 6GB VRAM single-GPU constraint, memory arithmetic dictates peak QLoRA footprints at approximately 3.0–3.3 GB (with sequence length $s=1024$, rank $r\le 32$), preserving a safe 0.5 GB margin above baseline OS overhead to prevent Out-Of-Memory (OOM) faults. All asynchronous workflows and background runners ensure zero event-loop starvation during intensive training or multi-seed evaluation passes.

Key Technical Highlights & Architecture

1. Hexagonal Architecture & Documentation (`PR #27`, `PR #28`)

* **`docs/lab-explained.md`**: Implements a comprehensive architectural guide mapping domain logic (`src/core/`) to infrastructure adapters (`src/adapters/`), safeguarded by automated boundary tests.

* **Execution Roadmap (`docs/plans/p3-p6-execution-plan.md` & `test_hexagonal_boundaries.py`)**: Establishes strict enforcement of domain isolation.

Code Snippet: Hexagonal Boundary Enforcement Test

backend/tests/unit/test_hexagonal_boundaries.py

from pathlib import Path

SOURCE_ROOT = Path(__file__).resolve().parents[2] / "src"

def _python_sources(folder: str) -> list[Path]:

```
return list((SOURCE_ROOT / folder).rglob("*.py"))
```

def test_core_has_no_adapter_or_bootstrap_dependencies():

```
forbidden = ("src.adapters", "src.bootstrap")
offenders = [
path.relative_to(SOURCE_ROOT).as_posix()
for path in _python_sources("core")
if any(token in path.read_text(encoding="utf-8") for token in forbidden)
]
assert offenders == []
```

```
---
```

2. Sandboxed Code Execution & Platform Hardening (`PR #26`)

* **Dual Execution Backends (`backend/src/adapters/evaluation/execution.py`)**: Provides `SubprocessCodeExecutor` for local environments and a production-grade `DockerSandboxExecutor` enforcing resource limits and isolation flags.

Diff Excerpt: Docker Sandbox Security Configuration

```
@@ -0,0 +1,128 @@
+DOCKER_SECURITY_FLAGS = [
+ "--network", "none",
+ "--memory", "512m",
+ "--cpus", "1.0",
+ "--pids-limit", "64",
+ "--read-only",
+ "--cap-drop", "ALL",
+]
+
+class DockerSandboxExecutor:
+ def __init__(self, image: str = "python:3.11-slim") -> None:
+ self.image = image
+
+ def __call__(self, code: str, timeout_seconds: float) -> tuple[bool, str]:
+ # Executes candidate payload in an ephemeral, isolated container boundary
+ ...
```

* **Asynchronous Experiment Runners (`backend/src/adapters/experiment_jobs.py`)**: Offloads long-running fine-tunes and multi-seed evaluations from synchronous HTTP workers using an `asyncio.Task` pool, supporting SSE and polling via `JobManager`.

Code Snippet: Async Task Submission (`ExperimentJobRunner`)

backend/src/adapters/experiment_jobs.py

class ExperimentJobRunner:

```
def __init__(self, *, jobs: JobManager, service: ExperimentService) -> None:
self.jobs = jobs
self.service = service
self._tasks: set[asyncio.Task] = set()

def submit_evaluation(self, request: ExperimentEvaluationRequest) -> Job:
job = self.jobs.create("evaluation")
task = asyncio.create_task(self._run_evaluation_job(job.id, request))
self._tasks.add(task)
task.add_done_callback(self._tasks.discard)
return job
```

* **HTTP-First MLflow with Local Failover (`backend/src/adapters/training/mlflow_tracker.py`)**: Configures tracking to prioritize a durable HTTP server (`http://127.0.0.1:5000`) with automatic SQLite fallback (`sqlite:///./data/mlflow.db`).

Code Snippet: Tracker Fallback Logic

backend/src/adapters/training/mlflow_tracker.py

def create_experiment_tracker(

```
    tracking_uri: str,
    experiment_name: str,
    fallback_tracking_uri: str | None = None,
```

) -> ExperimentTrackerPort:

```
    try:
    return MlflowExperimentTracker(tracking_uri=tracking_uri, experiment_name=experiment_name)
    except Exception as exc:
    if fallback_tracking_uri:
    return MlflowExperimentTracker(tracking_uri=fallback_tracking_uri,
    experiment_name=experiment_name)
    return UnavailableExperimentTracker(str(exc))
```

* **Statistical Rigor & Configuration Schema (`backend/src/bootstrap/config.py` & `multi_seed_evaluation.py`)**: Exposes validated execution configuration models and implements percentile bootstrap confidence intervals.

Code Snippet: Pydantic Evaluation Configuration Schema

backend/src/bootstrap/config.py

class AppConfig(BaseSettings):

```
    mlflow_tracking_uri: str = "http://127.0.0.1:5000"
    mlflow_tracking_fallback_uri: str = "sqlite:///./data/mlflow.db"
    eval_execution_backend: Literal["subprocess", "docker"] = "subprocess"
    eval_sandbox_memory_limit: str = "512m"
    eval_sandbox_pids_limit: int = 64
    ---
```

Risks, Tradeoffs, and Watch-Outs

* **Docker Daemon Requirement:** Setting `eval_execution_backend = "docker"` mandates an accessible Docker daemon. Fallback to host subprocess execution bypasses container isolation.

* **Network Isolation:** `--network none` prevents untrusted code from exfiltrating data or fetching packages during grading, meaning benchmarks requiring external assets must be pre-bundled.

* **VRAM Overhead:** Concurrent multi-seed evaluations must be explicitly throttled to respect the 6GB VRAM limit when loading model checkpoints.